САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа № 4

Выполнил:

Зарецкий Юрий

Группа
К3343

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service

Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом

Спроектируйте разделение вашей БД на обособленные БД

Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI

Опишите варианты запросов и ответов, задокументируйте возможные ошибки

В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разеделения вашего монолитного бэкенд-приложения на отдельные микросервисы

Ход работы

Разделил монолит на 3 микросервиса:

Сервис	Ответственность	Таблицы в БД
Identity Service	Пользователи, регистрация, аутентификация, выпуск JWT	users
Property & Rental Service	Справочники, каталог объявлений, фото, связь объект и удобства, сделки аренды	cities, comforts, properties, property_photos, property_comforts, rent_deals
Engagement Service	Переписки по объекту, сообщения, отзывы	conversations, messages, reviews

Разделение БД:

БД	Таблицы
identity_db	users
property_rental_db	cities, comforts, properties, property_photos, property_comforts, rent_deals
engagement_db	conversations, messages, reviews

В rent_deals поля tenant_id, owner_id ссылаются на пользователей логически; внешний ключ на identity_db не создаётся. В conversations / messages / reviews — user_id, property_id, без FK к чужим БД.

Взаимосвязь: Kafka как основной канал, API как экстренный

Топики:

Топик	**Producer**	**Потребители**	**Назначение**
identity.user.events	Identity	Engagement (и др.)	Пользователь создан/обновлён/деактивирован
rental.property.events	Property & Rental	Engagement	Объявление создано/обновлено/снято с публикации
rental.deal.events	Property & Rental	Engagement	Смена статуса сделки (для подсказок UI, ограничений отзыва «после аренды» и т.д.)

При необходимости можно объеденить в один топик rental.domain.events с полем eventType

Примеры событий:

identity.user.events - UserUpdated:

```
{
  "eventId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "occurredAt": "2026-03-31T12:00:00Z",
  "type": "user.updated",
  "payload": {
    "userId": 1,
    "email": "user@mail.com",
    "fullName": "Иван Иванов"
  }
}
```

rental.property.events - PropertyDeactivated:

```
{
  "eventId": "...",
  "occurredAt": "2026-03-31T12:05:00Z",
  "type": "property.deactivated",
  "payload": { "propertyId": 10, "ownerId": 3 }
}
```

rental.deal.events - DealApproved:

```
{
  "eventId": "...",
  "occurredAt": "2026-03-31T12:10:00Z",
  "type": "deal.approved",
  "payload": {
    "dealId": 42,
    "propertyId": 10,
    "tenantId": 1,
    "ownerId": 3,
    "startDate": "2026-04-01",
    "endDate": "2026-04-07"
  }
}
```

Взаимосвязь: API - только необходимый минимум

Доступ: сеть сервисов, path /internal, аутентификация service-to-service

Identity Service:

GET /internal/users/{id} - Нужен Engagement, чтобы в одном пользовательском запросе проверить, что пользователь существует и получить минимальный профиль без обращения к БД Identity

200: { "id": 1, "email": "user@mail.com", "fullName": "Иван Иванов"}

404: { "code": "NOT_FOUND", "message": "user not found" }

401: { "code": "UNAUTHORIZED", "message": "Invalid service credentials" }

GET /internal/users/exists?ids=1,2,3 - Нужен, чтобы за один round-trip проверить нескольких пользователей (например, оба участника чата или участников сделки), не делая серию вызовов GET /users/{id}

200: { "exists": { "1": true, "2": false, "3": true } }

400: неверный формат списка id

401: { "code": "UNAUTHORIZED", "message": "Invalid service credentials" }

Property & Rental Service:

GET /internal/properties/{id} - Нужен Engagement при создании беседы/отзыва, чтобы убедиться, что объект есть, активен и при необходимости совпадает владелец, не читая property_rental_db из чужого сервиса

200: {"id": 10, "title": "Студия у метро", "ownerId": 3, "cityId": 1, "isActive": true, "pricePerDay": 2500.0}

404: { "code": "NOT_FOUND", "message": "property not found" }

401: { "code": "UNAUTHORIZED", "message": "Invalid service credentials" }

GET /internal/rent-deals/{id} - Нужен Engagement, чтобы привязать отзыв или действие к конкретной одобренной сделке и за один запрос сверить участников и статус без своей копии таблицы сделок

200: { "id": 42, "propertyId": 10, "tenantId": 1, "ownerId": 3, "status": "approved" }

404: { "code": "NOT_FOUND", "message": "property not found" }

401: { "code": "UNAUTHORIZED", "message": "Invalid service credentials" }

Единый компонент ошибки в OpenAPI: components.schemas.Error с полями code, message

Вывод

В итоге получилось разобрать приложение на 3 микросервиса и продумать их взаимосвязь